

Практическое занятие 28: Работа с валидацией данных и защитой форм

Цель занятия:

- Научиться работать с валидацией данных в формах Django.
 - Реализовать дополнительные проверки и ограничения на вводимые данные.
 - Защитить формы от спама и автоматических отправок.
 - Понять, как обрабатывать ошибки валидации и отображать их пользователю.
-

Задачи занятия

1. Введение в валидацию данных и защиту форм
 2. Добавление пользовательской валидации в форму `ReviewForm`
 3. Реализация защиты формы от спама
 4. Обработка и отображение ошибок валидации
 5. Выполнение практического упражнения. Улучшение формы отзыва
 6. Выполнение дополнительных заданий. Реализация reCAPTCHA, ограничение количества отзывов
-

Задача 1: Введение в валидацию данных и защиту форм

1.1. Почему важна валидация данных?

- **Безопасность:** Предотвращение внедрения вредоносного кода и SQL-инъекций.
- **Корректность данных:** Обеспечение того, что данные соответствуют ожидаемому формату.
- **Улучшение UX:** Предоставление пользователю мгновенной обратной связи о неверно введенных данных.

1.2. Методы валидации в Django

- **Встроенные валидаторы:** Django предоставляет множество встроенных валидаторов, таких как `EmailValidator`, `MaxLengthValidator` и другие.

- **Пользовательские валидаторы:** Можно создавать собственные функции валидации.
- **Методы `clean` и `clean_field`:** Используются для валидации на уровне формы или отдельного поля.

1.3. Защита форм

- **CSRF-токен:** Защищает от Cross-Site Request Forgery атак.
- **Капча:** Предотвращает автоматическую отправку форм ботами.
- **Ограничение частоты отправки:** Предотвращает флуд и перегрузку сервера.

Задача 2: Добавление пользовательской валидации в форму `ReviewForm`

2.1. Проверка уникальности отзыва от пользователя

Задача: Запретить пользователю отправлять более одного отзыва для одного мероприятия с одним и тем же email.

Шаги:

1. Откройте файл `events/forms.py`.
2. Добавьте метод `clean_email` в класс `ReviewForm`:

```
# events/forms.py

from django import forms
from .models import Review

class ReviewForm(forms.ModelForm):
    class Meta:
        model = Review
        fields = ['name', 'email', 'rating', 'comment']
        widgets = {
            'rating': forms.Select(choices=[(i, i) for i in range(1,
6)]),
        }

    def __init__(self, *args, **kwargs):
        self.event = kwargs.pop('event', None)
        super().__init__(*args, **kwargs)

    def clean_email(self):
        email = self.cleaned_data.get('email')
```

```

        if Review.objects.filter(event=self.event,
email=email).exists():
            raise forms.ValidationError('Вы уже оставляли отзыв для
этого мероприятия.')
        return email

```

Пояснение:

- **Метод `__init__`**: Принимает дополнительный аргумент `event`, чтобы знать, для какого мероприятия проверять отзывы.
- **Метод `clean_email`**: Проверяет, есть ли уже отзыв от этого email для данного мероприятия.

3. Обновите представление `event_detail`, чтобы передавать `event` в форму:

```

# events/views.py

def event_detail(request, event_id):
    event = get_object_or_404(Event, id=event_id)
    reviews = event.reviews.all().order_by('-created_at')

    if request.method == 'POST':
        review_form = ReviewForm(request.POST, event=event)
        if review_form.is_valid():
            review = review_form.save(commit=False)
            review.event = event
            review.save()
            messages.success(request, 'Ваш отзыв успешно добавлен.')
            return redirect('events:event_detail', event_id=event.id)
        else:
            messages.error(request, 'Пожалуйста, исправьте ошибки в
форме.')
    else:
        review_form = ReviewForm(event=event)

    return render(request, 'events/event_detail.html', {
        'event': event,
        'reviews': reviews,
        'review_form': review_form,
    })

```

Пояснение:

- Передаем параметр `event` в форму при создании ее экземпляра.

2.2. Ограничение длины комментария

Задача: Ограничить максимальную длину комментария до 500 символов.

Шаги:

1. Добавьте валидатор к полю `comment` в модели `Review`:

```
# events/models.py

from django.core.validators import MaxLengthValidator

class Review(models.Model):
    # ... существующие поля ...
    comment = models.TextField(validators=[MaxLengthValidator(500)])
    # ... остальной код ...
```

2. Проверьте, что при вводе более 500 символов появляется ошибка валидации.

Задача 3: Реализация защиты формы от спама

3.1. Добавление скрытого поля (honeypot)

Задача: Использовать скрытое поле, которое боты заполняют, а реальные пользователи нет.

Шаги:

1. Добавьте скрытое поле в форму `ReviewForm`:

```
# events/forms.py

class ReviewForm(forms.ModelForm):
    honeypot = forms.CharField(required=False,
                               widget=forms.HiddenInput, label="Leave empty")

    class Meta:
        model = Review
        fields = ['name', 'email', 'rating', 'comment']
        widgets = {
            'rating': forms.Select(choices=[(i, i) for i in range(1,
6)]),
        }

    # ... остальной код ...
```

2. Добавьте проверку этого поля в методе `clean`:

```
# events/forms.py

def clean(self):
    cleaned_data = super().clean()
    honeypot = cleaned_data.get('honeypot')
    if honeypot:
        raise forms.ValidationError('Пожалуйста, оставьте это поле пустым.')
    return cleaned_data
```

Пояснение:

- Поле `honeypot` не отображается для пользователей, но боты могут его заполнить.
- Если поле заполнено, мы генерируем ошибку валидации.

3. Убедитесь, что поле не отображается в шаблоне:

```
<!-- events/templates/events/event_detail.html -->

<!-- ... ваш существующий код ... -->

<form method="post">
    {% csrf_token %}
    {{ review_form.non_field_errors }}
    {{ review_form.as_p }}
    <button type="submit">Отправить</button>
</form>
```

Пояснение:

- Поле с виджетом `HiddenInput` не будет отображаться в форме.

Задача 4: Обработка и отображение ошибок валидации

4.1. Отображение ошибок формы в шаблоне

Шаги:

1. Обновите форму в шаблоне `event_detail.html`, чтобы отображать ошибки:

```
<!-- events/templates/events/event_detail.html -->

<form method="post">
    {% csrf_token %}
    {{ review_form.non_field_errors }}
    <div>
```

```

        {{ review_form.name.label_tag }}
        {{ review_form.name }}
        {{ review_form.name.errors }}
    </div>
    <div>
        {{ review_form.email.label_tag }}
        {{ review_form.email }}
        {{ review_form.email.errors }}
    </div>
    <div>
        {{ review_form.rating.label_tag }}
        {{ review_form.rating }}
        {{ review_form.rating.errors }}
    </div>
    <div>
        {{ review_form.comment.label_tag }}
        {{ review_form.comment }}
        {{ review_form.comment.errors }}
    </div>
    <button type="submit">Отправить</button>
</form>

```

Пояснение:

- Отдельно отображаем ошибки для каждого поля, используя `{{ field.errors }}`.
- `non_field_errors` отображает ошибки, связанные с формой в целом.

4.2. Стилизация ошибок

Шаги:

1. Добавьте стили для отображения ошибок в ваш CSS-файл или внутри тега `<style>`:

```

<style>
.errorlist {
    color: red;
    list-style-type: none;
}
.errorlist li {
    margin-left: 0;
}
</style>

```

Пояснение:

- Стилизуем ошибки, чтобы они были заметны пользователю.

4.3. Обработка исключений в представлении

Шаги:

1. Обновите представление `event_detail`, чтобы обработать возможные исключения:

```
# events/views.py

from django.core.exceptions import ValidationError

def event_detail(request, event_id):
    # ... ваш существующий код ...

    if request.method == 'POST':
        review_form = ReviewForm(request.POST, event=event)
        try:
            if review_form.is_valid():
                review = review_form.save(commit=False)
                review.event = event
                review.save()
                messages.success(request, 'Ваш отзыв успешно
добавлен.')
                return redirect('events:event_detail',
event_id=event.id)
            else:
                messages.error(request, 'Пожалуйста, исправьте ошибки в
форме.')
        except ValidationError as e:
            review_form.add_error(None, e)
    else:
        review_form = ReviewForm(event=event)

    # ... остальной код ...
```

Пояснение:

- Обработываем исключения `ValidationError`, которые могут возникнуть при сохранении модели.
- Добавляем ошибки в форму, чтобы они отображались пользователю.

Задача 5: Практическое упражнение: Улучшение формы отзыва

Задание для студентов:

- Добавьте проверку на нецензурные слова в комментариях.

Подсказка: Создайте пользовательский валидатор для поля `comment`, который будет проверять наличие запрещенных слов.

- Убедитесь, что при вводе нецензурных слов появляется соответствующая ошибка.

Цель упражнения:

- Научиться создавать пользовательские валидаторы для полей формы.
- Закрепить навыки работы с валидацией данных и обработкой ошибок.

Шаг 1: Создание списка запрещенных слов

Примечание: Поскольку мы не можем использовать реальные нецензурные слова, вы можете создать свой список запрещенных слов в виде примера.

Пример:

```
# events/forms.py

PROFANITY_WORDS = ['badword1', 'badword2', 'badword3']
```

Пояснение:

- `PROFANITY_WORDS` — список слов, которые вы считаете нецензурными или нежелательными в комментариях.

Шаг 2: Создание пользовательского валидатора

Мы создадим функцию-валидатор, которая будет проверять наличие запрещенных слов в комментарии.

Код:

```
# events/forms.py

import re
from django.core.exceptions import ValidationError

def validate_no_profanity(value):
    for bad_word in PROFANITY_WORDS:
        if re.search(r'\b' + re.escape(bad_word) + r'\b', value,
re.IGNORECASE):
            raise ValidationError('Ваш комментарий содержит недопустимые
слова.')
```


Пояснение:

- Используем регулярные выражения для поиска запрещенных слов в тексте комментария.
- `re.escape(bad_word)` обеспечивает корректную обработку специальных символов.
- `\b` в регулярном выражении обозначает границы слова, чтобы избежать частичных совпадений.
- Если запрещенное слово найдено, вызывается `ValidationError`.

Шаг 3: Добавление валидатора к полю `comment` в форме

Код:

```
# events/forms.py

from django import forms
from .models import Review

PROFANITY_WORDS = ['badword1', 'badword2', 'badword3']

def validate_no_profanity(value):
    for bad_word in PROFANITY_WORDS:
        if re.search(r'\b' + re.escape(bad_word) + r'\b', value,
re.IGNORECASE):
            raise ValidationError('Ваш комментарий содержит недопустимые
слова.')

class ReviewForm(forms.ModelForm):
    # ... ваш существующий код ...

    class Meta:
        model = Review
        fields = ['name', 'email', 'rating', 'comment']
        widgets = {
            'rating': forms.Select(choices=[(i, i) for i in range(1, 6)]),
        }

    comment = forms.CharField(
        widget=forms.Textarea,
        validators=[validate_no_profanity],
        max_length=500,
        label='Комментарий'
    )

    # ... остальной код ...
```

Пояснение:

- Добавляем валидатор `validate_no_profanity` к полю `comment`.
- Определяем поле `comment` явно в форме, чтобы можно было добавить валидатор.
- Указываем `max_length=500`, если нужно ограничить длину комментария.

Шаг 4: Обновление обработки ошибок в шаблоне

Убедитесь, что ошибки валидации для поля `comment` отображаются пользователю.

Код:

```
<!-- events/templates/events/event_detail.html -->

<!-- ... ваш существующий код ... -->

<form method="post">
    {% csrf_token %}
    {{ review_form.non_field_errors }}
    <div>
        {{ review_form.name.label_tag }}
        {{ review_form.name }}
        {{ review_form.name.errors }}
    </div>
    <div>
        {{ review_form.email.label_tag }}
        {{ review_form.email }}
        {{ review_form.email.errors }}
    </div>
    <div>
        {{ review_form.rating.label_tag }}
        {{ review_form.rating }}
        {{ review_form.rating.errors }}
    </div>
    <div>
        {{ review_form.comment.label_tag }}
        {{ review_form.comment }}
        {{ review_form.comment.errors }}
    </div>
    <button type="submit">Отправить</button>
</form>
```

Пояснение:

- Обязательно выводим `{{ review_form.comment.errors }}`, чтобы пользователь видел сообщения об ошибках валидации комментария.

Шаг 5: Тестирование

1. Добавьте запрещенные слова в список `PROFANITY_WORDS`.

Например:

```
PROFANITY_WORDS = ['foo', 'bar', 'baz']
```

2. Попробуйте отправить комментарий, содержащий одно из запрещенных слов.
3. Убедитесь, что появляется сообщение об ошибке:

```
Ваш комментарий содержит недопустимые слова.
```

4. Попробуйте отправить комментарий без запрещенных слов.
5. Убедитесь, что отзыв успешно добавляется.

Полный код с учетом всех изменений

Файл `events/forms.py`

```
# events/forms.py

import re
from django import forms
from django.core.exceptions import ValidationError
from .models import Review

PROFANITY_WORDS = ['foo', 'bar', 'baz'] # Замените на свои запрещенные слова

def validate_no_profanity(value):
    for bad_word in PROFANITY_WORDS:
        if re.search(r'\b' + re.escape(bad_word) + r'\b', value,
re.IGNORECASE):
            raise ValidationError('Ваш комментарий содержит недопустимые слова.')
    return value

class ReviewForm(forms.ModelForm):
    honeypot = forms.CharField(required=False, widget=forms.HiddenInput,
label="Leave empty")

    comment = forms.CharField(
```

```

        widget=forms.Textarea,
        validators=[validate_no_profanity],
        max_length=500,
        label='Комментарий'
    )

class Meta:
    model = Review
    fields = ['name', 'email', 'rating', 'comment']
    widgets = {
        'rating': forms.Select(choices=[(i, i) for i in range(1, 6)]),
    }

def __init__(self, *args, **kwargs):
    self.event = kwargs.pop('event', None)
    super().__init__(*args, **kwargs)

def clean_email(self):
    email = self.cleaned_data.get('email')
    if Review.objects.filter(event=self.event, email=email).exists():
        raise forms.ValidationError('Вы уже оставляли отзыв для этого мероприятия.')
    return email

def clean(self):
    cleaned_data = super().clean()
    honeypot = cleaned_data.get('honeypot')
    if honeypot:
        raise forms.ValidationError('Пожалуйста, оставьте это поле пустым.')
    return cleaned_data

```

Файл `events/views.py`

Убедитесь, что представление передает ошибки формы в шаблон.

```

# events/views.py

from django.shortcuts import render, get_object_or_404, redirect
from django.contrib import messages
from django.core.exceptions import ValidationError
from .models import Event
from .forms import ReviewForm

def event_detail(request, event_id):
    event = get_object_or_404(Event, id=event_id)
    reviews = event.reviews.all().order_by('-created_at')

    if request.method == 'POST':

```

```

review_form = ReviewForm(request.POST, event=event)
try:
    if review_form.is_valid():
        review = review_form.save(commit=False)
        review.event = event
        review.save()
        messages.success(request, 'Ваш отзыв успешно добавлен.')
        return redirect('events:event_detail', event_id=event.id)
    else:
        messages.error(request, 'Пожалуйста, исправьте ошибки в
форме.')
except ValidationError as e:
    review_form.add_error(None, e)
else:
    review_form = ReviewForm(event=event)

return render(request, 'events/event_detail.html', {
    'event': event,
    'reviews': reviews,
    'review_form': review_form,
})

```

Файл `events/templates/events/event_detail.html`

```

<!-- events/templates/events/event_detail.html -->

{% extends 'base.html' %}

{% block title %}{{ event.title }}{% endblock %}

{% block content %}
    <h1>{{ event.title }}</h1>
    <!-- ... остальной контент мероприятия ... -->

    <h2>Отзывы</h2>
    {% if reviews %}
        {% for review in reviews %}
            <div>
                <p><strong>{{ review.name }}</strong> - Оценка: {{
review.rating }}/5</p>
                <p>{{ review.comment }}</p>
                <p><em>{{ review.created_at|date:"d.m.Y H:i" }}</em></p>
            </div>
        {% endfor %}
    {% else %}
        <p>Нет отзывов о данном мероприятии.</p>
    {% endif %}

```

```
<!-- Форма добавления отзыва -->
<h2>Оставить отзыв</h2>
<form method="post">
    {% csrf_token %}
    {{ review_form.non_field_errors }}
    <div>
        {{ review_form.name.label_tag }}
        {{ review_form.name }}
        {{ review_form.name.errors }}
    </div>
    <div>
        {{ review_form.email.label_tag }}
        {{ review_form.email }}
        {{ review_form.email.errors }}
    </div>
    <div>
        {{ review_form.rating.label_tag }}
        {{ review_form.rating }}
        {{ review_form.rating.errors }}
    </div>
    <div>
        {{ review_form.comment.label_tag }}
        {{ review_form.comment }}
        {{ review_form.comment.errors }}
    </div>
    <button type="submit">Отправить</button>
</form>
{% endblock %}
```

Дополнительные рекомендации

- **Расширение списка запрещенных слов:**
 - Вы можете загрузить список запрещенных слов из файла или использовать сторонние библиотеки для фильтрации нецензурной лексики.
- **Использование сторонних пакетов:**
 - Есть готовые библиотеки для фильтрации нецензурных слов, например, `django-profanity-check` или `profanity-filter`.
 - Установка:

```
pip install profanity-filter
```

- Использование:

```
from profanity_filter import ProfanityFilter

pf = ProfanityFilter()

def validate_no_profanity(value):
    if pf.is_profane(value):
        raise ValidationError('Ваш комментарий содержит
недопустимые слова.')
    return value
```

- **Учтите варианты написания слов:**

- Пользователи могут обходить фильтр, заменяя буквы на похожие символы или пропуская буквы.
- Более сложные алгоритмы и библиотеки могут помочь в таких случаях.

Вы успешно реализовали проверку на нецензурные слова в комментариях, создав пользовательский валидатор для поля `comment`. Теперь при вводе запрещенных слов пользователи будут получать сообщение об ошибке, и отзыв не будет сохранен до исправления комментария.

Дополнительное задание

1. Реализация проверки reCAPTCHA

Цель: Добавить защиту формы с использованием Google reCAPTCHA.

Шаги:

1. **Зарегистрируйтесь на сайте [Google reCAPTCHA](#)** и получите ключи сайта и секретный ключ.
2. **Установите пакет `django-recaptcha`:**

```
pip install django-recaptcha
```

3. **Добавьте `captcha` в `INSTALLED_APPS` в `settings.py`:**

```
# settings.py

INSTALLED_APPS = [
    # ... другие приложения ...
```

```
        'captcha',  
    ]
```

4. Добавьте ключи reCAPTCHA в настройки:

```
# settings.py  
  
RECAPTCHA_PUBLIC_KEY = 'ваш_публичный_ключ'  
RECAPTCHA_PRIVATE_KEY = 'ваш_секретный_ключ'  
RECAPTCHA_USE_SSL = True
```

5. Обновите форму ReviewForm:

```
# events/forms.py  
  
from captcha.fields import ReCaptchaField  
from captcha.widgets import ReCaptchaV2Checkbox  
  
class ReviewForm(forms.ModelForm):  
    captcha = ReCaptchaField(widget=ReCaptchaV2Checkbox)  
  
    class Meta:  
        model = Review  
        fields = ['name', 'email', 'rating', 'comment', 'captcha']  
        widgets = {  
            'rating': forms.Select(choices=[(i, i) for i in range(1,  
6)]),  
        }  
  
# ... остальной код ...
```

6. Проверьте работу формы и убедитесь, что reCAPTCHA отображается и работает корректно.

2. Ограничение количества отзывов от одного пользователя в день

Цель: Предотвратить спам, ограничив количество отзывов, которые пользователь может оставить за день.

Шаги:

1. Добавьте проверку в методе `clean_email` формы ReviewForm:

```
# events/forms.py
```



```
from django.utils import timezone
from datetime import timedelta

def clean_email(self):
    email = self.cleaned_data.get('email')
    time_threshold = timezone.now() - timedelta(days=1)
    reviews_count = Review.objects.filter(email=email,
created_at__gte=time_threshold).count()
    if reviews_count >= 5:
        raise forms.ValidationError('Вы превысили лимит отзывов на
сегодня.')
    return email
```

Пояснение:

- Проверяем количество отзывов, оставленных с данного email за последние 24 часа.
- Устанавливаем лимит, например, 5 отзывов в день.

Заключение

Поздравляем! Вы успешно:

- Научились добавлять пользовательскую валидацию в формы Django.
- Реализовали защиту форм от спама и автоматических отправок.
- Научились обрабатывать и отображать ошибки валидации для пользователей.

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться.

Успехов в разработке!

Вопросы для обсуждения

- Почему важно проводить валидацию данных как на стороне клиента, так и на стороне сервера?
- Валидация данных на стороне клиента обеспечивает мгновенную обратную связь пользователю и уменьшает нагрузку на сервер, предотвращая отправку некорректных данных. Однако она может быть обойдена злоумышленниками, поэтому валидация на стороне сервера необходима для обеспечения

безопасности и целостности данных, гарантируя, что все входящие данные проверены и соответствуют требованиям приложения.

- **Какие еще методы защиты форм вы можете предложить?**
 - Помимо валидации данных и использования капчи, можно применять следующие методы защиты форм: ограничение частоты отправки форм (rate limiting), использование токенов CSRF для предотвращения межсайтовых запросов, проверка заголовков HTTP-запросов, применение скрытых полей (honeypot), шифрование чувствительных данных, а также использование безопасных протоколов передачи данных, таких как HTTPS.
 - **Как обеспечить баланс между удобством использования формы и ее безопасностью?**
 - Для обеспечения баланса между удобством и безопасностью формы следует внедрять интуитивно понятные интерфейсы с четкой структурой и обратной связью, избегать излишней сложности для пользователя, одновременно применяя необходимые меры безопасности, такие как валидация и защита от CSRF. Использование вспомогательных инструментов, таких как всплывающие подсказки и авто-заполнение, также может повысить удобство без ущерба для безопасности. Важно проводить тестирование UX и безопасности, чтобы найти оптимальный баланс, удовлетворяющий как пользователей, так и требования безопасности.
-

Полезные ресурсы

- [Документация Django: Валидация форм](#)
- [Документация Django: Валидаторы](#)
- [django-recaptcha](#)
- [django-ratelimit](#)